

```
=====
Spring ORM
=====
```

=> Spring ORM is a module in the Spring Framework that provides integration layers for popular ORM frameworks such as:

- 1) Hiberante
- 2) JPA
- 3) IBatis

=> It allows developers to interact with relational databases using objects instead of SQL queries.

=> ORM stands for Object Relational Mapping

=> Using ORM Principles we can perform DB operations using java objects directley.

Ex:

- a) Insert Object (one row will be inserted in table)
- b) Retrieve Object (one row will be retrieved from table)
- c) Delete Object (one row will be deleted from table)
- d) Update object (one row will be updated in table)

=> When we use ORM, We no need to write SQL queries.. ORM framework will generate SQL queries for us.

=> In Spring ORM we have pre-defined components to perform DB operations using Objects

Ex: HibernateTemplate

```
=====
Spring ORM Important Components
=====
```

- 1) DataSource
- 2) SessionFactory
- 3) Session
- 4) HibernateTransactionManager
- 5) HibernateTemplate
- 6) Entity Class

DataSource :: Provides the database connection information. It Manages connections with properties like URL, username, password, and driverClass.

=> DriverManagerDataSource class is impl class of DataSource.

...

```
@Bean
public DataSource dataSource() {

    DriverManagerDataSource ds = new DriverManagerDataSource();

    ds.setDriverClassName("com.mysql.cj.jdbc.Driver");
    ds.setUrl("jdbc:mysql://localhost:3306/sbms75");
```

```

        ds.setUsername("root");
        ds.setPassword("root");

        return ds;
    }
}

```

SessionFactory :: It Holds metadata about the database and ORM mappings. Heavy-weight object and should be created once and shared with all the methods.

```

        hbm2ddl.auto = update (It create tables in db)

        show_sql = true (It prints generated queries on console)

        format_sql = true (print queries in proper format)

        dialect = org.hibernate.dialect.MySQL8Dialect (converts HQL into SQL)

```

=> SessionFactory interface implementing by LocalSessionFactoryBean class

```

}
}

@Bean
public LocalSessionFactoryBean sessionFactory() {

    LocalSessionFactoryBean factory = new LocalSessionFactoryBean();

    factory.setDataSource(dataSource());

    factory.setPackagesToScan("in.ashokit.entity");

    Properties props = new Properties();
    props.put("hibernate.dialect", "org.hibernate.dialect.MySQL8Dialect");
    props.put("hibernate.hbm2ddl.auto", "update");
    props.put("hibernate.show_sql", "true");
    props.put("hibernate.format_sql", "true");

    factory.setHibernateProperties(props);

    return factory;
}
}

```

Session : Represents a single unit of work with the database. It is obtained from SessionFactory.

HibernateTransactionManager : A Spring-provided transaction manager for Hibernate. Manages transactions declaratively (via @Transactional) or programmatically.

DML : Transaction Required

HibernateTemplate : Helper class provided by Spring.

=> Simplifies Hibernate data access by handling boilerplate code like opening/closing sessions and catching exceptions.

```

}
}

@Repository
@Transactional
public class StudentDao {

    @Autowired
    private HibernateTemplate hibernateTemplate;

    public void saveStudent(Student student) {
        hibernateTemplate.save(student);
    }
}

```

```

    public List<Student> getAllStudents() {
        return hibernateTemplate.loadAll(Student.class);
    }
}

```

Entity Class : The java class which is mapped with Database Table.

Java class name -----> Table name

class variables -----> table column names

class object -----> Table row

-> To map our java class with DB table we will use below annotations

- a) @Entity
- b) @Table
- c) @Id
- d) @Column

Note: @Table and @Column annotations are optional.

```

=====
Spring ORM Application Development
=====

```

1) Create Maven Project and add below dependencies

- a) spring-context
- b) spring-orm
- c) spring-tx
- d) hiberante-core
- e) mysql-connector-j
- f) project-lombok

2) Create Entity Class (Object Relational Mapping)

3) Create DAO class to perform DB Operations using HibernateTemplate class

4) Create Service class to call DAO methods

5) Create Config class and configure required spring beans

Ex: DataSource, LocalSessionFactoryBean, HiberanteTxManager, HibernateTemplate..

6) Create Main class to test the application.

```

=====
HibernateTemplate class methods
=====

```

=> HibernateTemplate is a helper class in Spring ORM that simplifies Hibernate data access.

=> It reduces boilerplate code for opening/closing sessions, transactions, and exception handling.

=> Below are the methods available in HiberanteTemplate class

- 1) save(Object entity)
- 2) update(Object entity)
- 3) saveOrUpdate(Object entity)
- 4) delete(Object entity)

- 5) get(Class entityClass, Serializable id)
- 6) load(Class entityClass, Serializble id)
- 7) loadAll(Class class)
- 8) find(String query, Object... values)